ChatGPT Conversation:

Me:
for my car garage web app, i cant figure out why cars are being appended multiple times and show up multiple times on the page.

```
from drafter import *
from drafter import Picture
from dataclasses import dataclass
@dataclass
class State:
    garage: list[Car]
    drawn_garage: list[Picture]
    latest_car: Car
    drawn_car: Picture
    car_color: str
@dataclass
class Car:
    body: list[str]
    color: str
MATERIALS = {
    "B": "body",
    "T": "tire",
    "W": "window"
    }
COLORS = {
    "Blue": "b",
    "Red": "r",
    "Green": "g",
    "Orange": "o",
    "Pink": "p",
    "Black": "x"
    }

CARS = {
"sports": [
    "     BBBBBBBBBBBWWWWWW     ",
    "    BBBBBBBBBBBBWWWWWWW",
    "BBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
    "BBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
    "BBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
    "BBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
    "  TTTTTT              TTTTTT"
    ],
"truck": [
```

```
    "        BBBBBBBB",
    "        BBBBBWWWW",
    "        BBBBBWWWWW",
    "BBBBBBBBBBBBBBBBBBBBBBB",
    "BBBBBBBBBBBBBBBBBBBBBBB",
    "BBBBBBBBBBBBBBBBBBBBBBB",
    "  TTTT           TTTT"
    ],
"van": [
    "BBBBBBBBBBBBBBBBBBBBB",
    "BBBBBBBBBBBBBBBBBBWWWWW",
    "BBBBBBBBBBBBBBBBBBWWWWWW",
    "BBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
    "BBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
    "BBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
    "  TTTTT              TTTTT"
    ]
}

pix_ht = 10
pix_wd = 10

@route
def index(state: State)->Page:
    content = [
        Header("Shitty Pixel Car Garage"),
        "Choose a shit-box\n",
        Button("Vroom Car", "make_sports"),
        Button("Truk","make_truck"),
        Button("Mom van","make_van")
        ]
    for car_str in state.garage:
        content.append(color_car(car_str,car_str.color))
    for i,item in enumerate(content):
        if content[i-1] == "Choose a shit-box\n":
            i = i + 1
            content[2],content[-1] = content[-1],content[2]

    return Page(
        state, content)
@route
def make_sports(state: State):
    state.latest_car = Car(CARS["sports"],"")
    state.garage.append(state.latest_car)
```

```
    return Page(
        state, [
        "Choose a body paint color",
        RadioButtonGroup("car_color", ["Green","Blue","Red","Pink","Black","Orange"]),
        Button("Apply","choose_color")
        ])
@route
def make_truck(state: State):
    state.latest_car = Car(CARS["truck"],"")
    state.garage.append(state.latest_car)
    return Page(
        state, [
        "Choose a body paint color",
        RadioButtonGroup("car_color", ["Green","Blue","Red","Pink","Black","Orange"]),
        Button("Apply","choose_color")
        ])
@route
def make_van(state: State):
    state.latest_car = Car(CARS["van"],"")
    state.garage.append(state.latest_car)
    return Page(
        state, [
        "Choose a body paint color",
        RadioButtonGroup("car_color", ["Green","Blue","Red","Pink","Black","Orange"]),
        Button("Apply","choose_color")
        ])
@route
def choose_color(state: State, car_color:str):
    state.car_color = car_color
    state.latest_car.color = car_color
    state.drawn_car = color_car(state.latest_car,car_color)
    if state.latest_car.body == CARS["sports"]:
        return Page(
            state, [
            "NeEEeEEoOWWWWW\n",
            state.drawn_car,
            Button("Save to garage","add_to_garage")
            ])
    elif state.latest_car.body == CARS["truck"]:
        return Page(
            state, [
            "now that's a TRUK\n",
            state.drawn_car,
            Button("Save to garage","add_to_garage")
```

```
            ])
    else:
        return Page(
            state, [
            "wow so hip\n",
            state.drawn_car,
            Button("Save to garage","add_to_garage")
            ])
scale = 20
def color_car(car:Car,car_color:str):
    car_canvas = Picture.new(800,300)
    for y, row in enumerate(car.body):
        for x, pixel in enumerate(row):
            for dy in range(scale):
                for dx in range(scale):
                    if pixel == "B":
                        car_canvas.set_pixel(x*scale+dx,y*scale+dy,car_color)
                    elif pixel == "W":
                        car_canvas.set_pixel(x*scale+dx,y*scale+dy,"gray")
                    elif pixel == "T":
                        car_canvas.set_pixel(x*scale+dx,y*scale+dy,"black")
                    else:
                        pass
    return car_canvas
@route
def add_to_garage(state: State):
    #color = state.latest_car.color
    state.garage.append(state.latest_car)
    return index(state)
#def make_viewable(car_pic: Picture):
#    pass
start_server(State([],None,[],None,""))
```

Chat:

I found the problem! You're appending each car to `state.garage` in **two different places**. That's why you're seeing duplicates.

1. The first append happens in `make_sports`, `make_truck`, and `make_van`

For example:

```
None
@route
def make_sports(state: State):
    state.latest_car = Car(CARS["sports"], "")
    state.garage.append(state.latest_car)  # FIRST APPEND
```

You're adding the car to the garage as soon as the user selects its model.

2. The second append happens in `add_to_garage`

```
None
@route
def add_to_garage(state: State):
    state.garage.append(state.latest_car)  # SECOND APPEND
    return index(state)
```

When the user clicks "Save to garage," you're adding that exact same car to the garage again.

So the same car gets added twice.

3. How I would fix it

Think about the intended sequence:

1. User selects a car model.
2. The car is created and stored in `state.latest_car`.
3. User selects a color.
4. User previews the car.
5. User clicks "Save to garage."
6. The car gets appended to `state.garage`.

With that in mind, you should only append the car in `add_to_garage`.

Remove the `state.garage.append(state.latest_car)` line from all three `make_` functions.

4. One other thing I noticed in your code

In `index`, you have:

```
None
for car_str in state.garage:
    content.append(color_car(car_str, car_str.color))
```

This is actually fine logically, but there's something worth considering:

You have a `drawn_garage: list[Picture]` in your State dataclass, but you're not currently using it.

You don't necessarily need it! Since `color_car()` generates a picture, you can generate the pictures as you build the garage page.

Also, your `index` function has this section:

```
None
for i,item in enumerate(content):
    if content[i-1] == "Choose a shit-box\n":
        i = i + 1
        content[2],content[-1] = content[-1],content[2]
```

I'd recommend temporarily removing this entire section while debugging. It isn't necessary for displaying the garage and makes it harder to track what's happening to your page contents.

My recommendation: Fix the double append first, then test adding one car, followed by two different cars. That should help isolate whether anything else is causing duplicates.

Me:
i want all the cars i add to the front page to show up under the buttons in index after they are asaved

Chat:

Yes! That's exactly what your `index()` function should be doing. You are already very close.

The goal is:

1. The user selects a car model.
2. The user chooses its color.
3. The user previews it.
4. The user clicks **Save to garage**.
5. The car is appended to `state.garage`.
6. The app returns to `index()`, which displays every saved car underneath the buttons.

1. Your `add_to_garage()` function

This part is already set up correctly:

```
None
@route
def add_to_garage(state: State):
    state.garage.append(state.latest_car)
    return index(state)
```

This saves the car and returns the user to the front page.

2. Your `index()` function

You already have the right idea here:

```
None
for car_str in state.garage:
    content.append(color_car(car_str, car_str.color))
```

This loops through every saved car and creates its picture.

However, I'd simplify the rest of your `index()` function to make sure the cars appear after the buttons.

For example:

```
None
@route
def index(state: State)->Page:
    content = [
        Header("Shitty Pixel Car Garage"),
        "Choose a shit-box\n",
        Button("Vroom Car", "make_sports"),
        Button("Truk", "make_truck"),
        Button("Mom van", "make_van")
    ]
```

```
    for car in state.garage:
        content.append(color_car(car, car.color))

    return Page(state, content)
```

Notice that I've removed your `enumerate` loop and the swapping of `content[2]` and `content[-1]`.

3. One important distinction

You have two separate concepts:

- `state.latest_car`: The car currently being customized.
- `state.garage`: All the cars the user has saved.

You want to add a car to `state.garage` **only when the user clicks Save to garage**.

That way, returning to `index()` displays all previously saved cars, without adding the current car again.

4. Test it

Try this sequence:

- Add a blue sports car.
- Save it.
- Add a red truck.
- Save it.
- Add a green van.
- Save it.

Your front page should show all three cars underneath the buttons.

One question to consider for your next step: **Do you want the cars to remain in the garage if the user goes back and adds another car, or do you eventually want a button to remove cars from the garage, too?**

Me: this is the screen after making a blue sports car and a red truck

Chat:

Okay, the screenshot helps! Both cars are being saved, and both are appearing on the front page. So your saving mechanism is working.

However, there's one remaining issue: **the blue sports car is appearing above the buttons instead of below them.** The red truck is appearing in the correct general area.

I suspect this is related to the code you previously had in `index()`:

```
None
for i,item in enumerate(content):
    if content[i-1] == "Choose a shit-box\n":
        i = i + 1
        content[2],content[-1] = content[-1],content[2]
```

This code manually rearranges the contents of your page. The problem is that it swaps elements based on their positions, rather than consistently placing cars after the buttons.

What I want you to try

1. Remove that entire `for i, item in enumerate(content)` loop.
2. Keep your garage loop immediately after the three buttons:

```
None
for car in state.garage:
    content.append(color_car(car, car.color))
```

3. Return the page normally:

```
None
return Page(state, content)
```

Your intended order should be:

Shitty Pixel Car Garage

Choose a shit-box

Vroom Car
Truk
Mom van
[Race Car Sticker by BoxMedia - GIPHY]

pinterest.com
Red pixel pickup icon. Side view. Vector graphic illustration. Isolated object on a white background. Isolate.
All saved cars appear underneath the buttons, in the order they were added.

One question: Have you already removed that rearranging loop from your actual code, or is it still there? That will help me narrow down exactly why the first car is appearing above the buttons.

Me: ive already removed it

Chat: Okay, then let's narrow this down. The screenshot tells me something important: **the two cars are being displayed in different positions relative to the buttons**, even though both should be coming from the same `state.garage` loop.

I don't want to guess and have you change something that isn't broken.

Could you paste your **current, updated** `index()` **function** exactly as it is now?

I want to check two things:

1. Where the garage loop is positioned relative to the buttons.
2. Whether anything else is adding a picture to `content` before the buttons.

We'll figure out why the first car is behaving differently from the second without rewriting the rest of your app.

Me:
whole code:

```
from drafter import *
from drafter import Picture
from dataclasses import dataclass
@dataclass
class State:
    garage: list[Car]
    drawn_garage: list[Picture]
    latest_car: Car
    drawn_car: Picture
    car_color: str
@dataclass
class Car:
    body: list[str]
    color: str
MATERIALS = {
    "B": "body",
```

```
    "T": "tire",
    "W": "window"
    }
COLORS = {
    "Blue": "b",
    "Red": "r",
    "Green": "g",
    "Orange": "o",
    "Pink": "p",
    "Black": "x"
    }

CARS = {
"sports": [
    "     BBBBBBBBBBBWWWWWW     ",
    "    BBBBBBBBBBBBWWWWWWW",
    "BBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
    "BBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
    "BBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
    "BBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
    "  TTTTTT              TTTTTT"
    ],
"truck": [
    "         BBBBBBBB",
    "         BBBBBWWWW",
    "         BBBBBWWWWW",
    "BBBBBBBBBBBBBBBBBBBBBBB",
    "BBBBBBBBBBBBBBBBBBBBBBB",
    "BBBBBBBBBBBBBBBBBBBBBBB",
    "  TTTT            TTTT"
    ],
"van": [
    "BBBBBBBBBBBBBBBBBBBBB",
    "BBBBBBBBBBBBBBBBBBWWWWW",
    "BBBBBBBBBBBBBBBBBBWWWWWW",
    "BBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
    "BBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
    "BBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
    "  TTTTT               TTTTT"
    ]
}

pix_ht = 10
pix_wd = 10
```

```
@route
def index(state: State)->Page:
    content = [
        Header("Shitty Pixel Car Garage"),
        "Choose a shit-box\n",
        Button("Vroom Car", "make_sports"),
        Button("Truk","make_truck"),
        Button("Mom van","make_van")
        ]
    for car in state.garage:
        content.append(color_car(car,car.color))
    return Page(
        state, content)
@route
def make_sports(state: State):
    state.latest_car = Car(CARS["sports"],"")
    return Page(
        state, [
        "Choose a body paint color",
        RadioButtonGroup("car_color", ["Green","Blue","Red","Pink","Black","Orange"]),
        Button("Apply","choose_color")
        ])
@route
def make_truck(state: State):
    state.latest_car = Car(CARS["truck"],"")
    return Page(
        state, [
        "Choose a body paint color",
        RadioButtonGroup("car_color", ["Green","Blue","Red","Pink","Black","Orange"]),
        Button("Apply","choose_color")
        ])
@route
def make_van(state: State):
    state.latest_car = Car(CARS["van"],"")
    return Page(
        state, [
        "Choose a body paint color",
        RadioButtonGroup("car_color", ["Green","Blue","Red","Pink","Black","Orange"]),
        Button("Apply","choose_color")
        ])
@route
def choose_color(state: State, car_color:str):
    state.car_color = car_color
```

```
    state.latest_car.color = car_color
    state.drawn_car = color_car(state.latest_car,car_color)
    if state.latest_car.body == CARS["sports"]:
        return Page(
            state, [
            "NeEEeEEoOWWWWW\n",
            state.drawn_car,
            Button("Save to garage","add_to_garage")
            ])
    elif state.latest_car.body == CARS["truck"]:
        return Page(
            state, [
            "now that's a TRUK\n",
            state.drawn_car,
            Button("Save to garage","add_to_garage")
            ])
    else:
        return Page(
            state, [
            "wow so hip\n",
            state.drawn_car,
            Button("Save to garage","add_to_garage")
            ])
scale = 20
def color_car(car:Car,car_color:str):
    car_canvas = Picture.new(800,300)
    for y, row in enumerate(car.body):
        for x, pixel in enumerate(row):
            for dy in range(scale):
                for dx in range(scale):
                    if pixel == "B":
                        car_canvas.set_pixel(x*scale+dx,y*scale+dy,car_color)
                    elif pixel == "W":
                        car_canvas.set_pixel(x*scale+dx,y*scale+dy,"gray")
                    elif pixel == "T":
                        car_canvas.set_pixel(x*scale+dx,y*scale+dy,"black")
                    else:
                        pass
    return car_canvas
@route
def add_to_garage(state: State):
    #color = state.latest_car.color
    state.garage.append(state.latest_car)
    return index(state)
```

#def make_viewable(car_pic: Picture):
pass
start_server(State([],None,[],None,""))

Chat:

Now that I can see the entire updated code, I can narrow this down considerably.

Your `index()` function is logically correct. In fact, based on the code you've posted, both cars should appear underneath the three buttons.

Let's look at the order you're building the page:

```
None
content = [
    Header("Shitty Pixel Car Garage"),
    "Choose a shit-box\n",
    Button("Vroom Car", "make_sports"),
    Button("Truk","make_truck"),
    Button("Mom van","make_van")
]

for car in state.garage:
    content.append(color_car(car,car.color))
```

The order should be:

1. Header
2. Choose a shit-box
3. Vroom Car button
4. Truk button
5. Mom van button
6. Blue sports car
7. Red truck

There is nothing in this function that moves the first car above the buttons.

Here's what I want to investigate

I suspect we need to determine whether this is a problem with how Drafter displays `Picture` objects or whether something else is happening when the page is generated.

Try temporarily changing your loop to:

```
None
for i, car in enumerate(state.garage):
    content.append(f"Car number {i+1}:")
    content.append(color_car(car, car.color))
```

This will put a text label immediately before each saved car.

Then look at the website.

- If the labels and cars are all underneath the buttons, everything is working as expected.
- If the labels are underneath the buttons but the first picture is above them, we have isolated the problem to picture rendering.
- If the first label is also above the buttons, something else is affecting the page layout or the code being executed.

One other observation

You currently have:

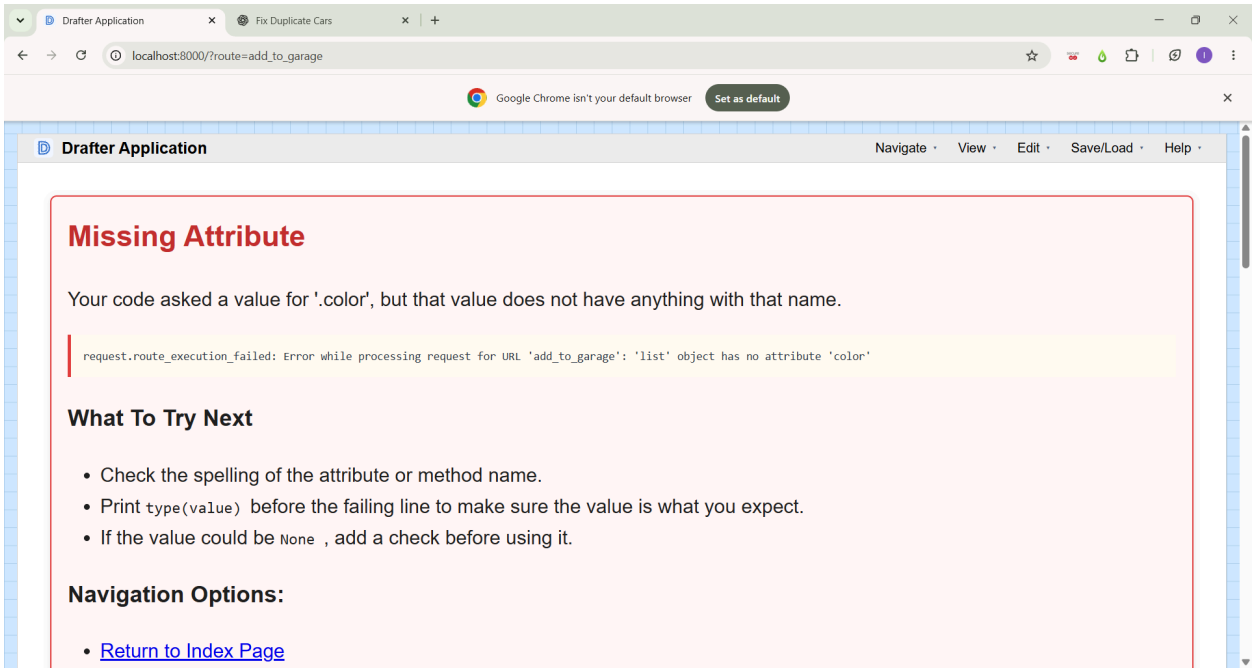
```
None
drawn_garage: list[Picture]
```

in your State dataclass, but you aren't using it anywhere. That's perfectly fine; you don't need to maintain a separate list of pictures when you can generate them from `state.garage`.

I would leave that alone for now.

Let's try the numbered labels first. I don't want to change your drawing function or your saving mechanism when those aren't showing an obvious problem.

Me:



Chat:

Ah! This error gives us a much better clue. I think we've found the issue.

The important part of the error is:

```
None
'list' object has no attribute 'color'
```

This means that somewhere in `state.garage`, Python is finding a **list instead of a** `Car` **object**.

1. Look at your State initialization

At the bottom of your code, you have:

```
None
start_server(State([],None,[],None,""))
```

Compare that with your State dataclass:

```
None
@dataclass
class State:
    garage: list[Car]
    drawn_garage: list[Picture]
    latest_car: Car
    drawn_car: Picture
    car_color: str
```

Your initial values are currently:

Attribute	Your value	Expected
garage	`[]`	`[]`
drawn_garage	`None`	`[]`
latest_car	`[]`	`None`
drawn_car	`None`	`None`
car_color	`""`	`""`

The third value is particularly important: `latest_car` should not initially be a list.

Change your initialization to:

```
None
start_server(State([], [], None, None, ""))
```

2. Let's investigate the actual contents of your garage

Inside `index()`, temporarily add:

```
None
for car in state.garage:
    print(type(car))
    print(car)
    content.append(color_car(car, car.color))
```

Ideally, you should see something like:

```
None
<class '__main__.Car'>
Car(body=[...], color='Blue')
<class '__main__.Car'>
Car(body=[...], color='Red')
```

If you see `<class 'list'>`, we've confirmed that a list is being stored in your garage instead of a `Car` object.

3. Important: Restart your server

After changing the initialization, stop your Drafter server completely and start it again.

This ensures you're starting with a fresh State rather than potentially carrying over data from an earlier version of your program.

Let's do these two things first: fix the initialization and print the types of the objects in your garage.

I suspect the problem is related to the objects being stored in `state.garage`, rather than the way you're displaying them.

Me:

Console:
Loaded Jinja2, MarkupSafe, contourpy, cycler, fonttools, kiwisolver, matplotlib, numpy, packaging, pyparsing, python-dateutil, pytz, six <class '__main__.Car'> Car(body=[' BBBBBBBB', ' BBBBBWWWW', ' BBBBBWWWWW', 'BBBBBBBBBBBBBBBBBBBBBBBB', 'BBBBBBBBBBBBBBBBBBBBBBBB', 'BBBBBBBBBBBBBBBBBBBBBBBB', ' TTTT TTTT'], color='Green') <class '__main__.Car'> Car(body=[' BBBBBBBB', ' BBBBBWWWW', ' BBBBBWWWWW', 'BBBBBBBBBBBBBBBBBBBBBBBB', 'BBBBBBBBBBBBBBBBBBBBBBBB', 'BBBBBBBBBBBBBBBBBBBBBBBB', ' TTTT

TTTT'], color='Green') <class '__main__.Car'> Car(body=['BBBBBBBBBBBBBBBBBBBB',
'BBBBBBBBBBBBBBBBBBWWWWW', 'BBBBBBBBBBBBBBBBBBWWWWWW',
'BBBBBBBBBBBBBBBBBBBBBBBBBBBBB',
'BBBBBBBBBBBBBBBBBBBBBBBBBBBBB',
'BBBBBBBBBBBBBBBBBBBBBBBBBBBBB', ' TTTTT   TTTTT'], color='Blue')
>>>

Chat:

Great! This output actually tells us quite a bit. We've confirmed that **your garage contains `Car` objects, not lists.** So the `.color` error is resolved.

However, I noticed something interesting in your console output.

1. Your garage currently contains three cars

Here's what Python is reporting:

Position	Car	Color
1	Truck	Green
2	Truck	Green
3	Van	Blue

The first two entries are identical.

This tells us that there are actually two separate issues worth investigating:

- The same truck is being saved twice.
- The first picture is appearing above the buttons instead of below them.

2. Let's isolate the display problem first

I want you to try something slightly different in `index()`.

Change your garage loop to:

None

```
for i, car in enumerate(state.garage):
    content.append(f"CAR NUMBER {i+1}")
    content.append(color_car(car, car.color))
```

This will help us determine whether Drafter is moving the pictures or whether the entire content sequence is being rearranged.

3. Something else I want you to check

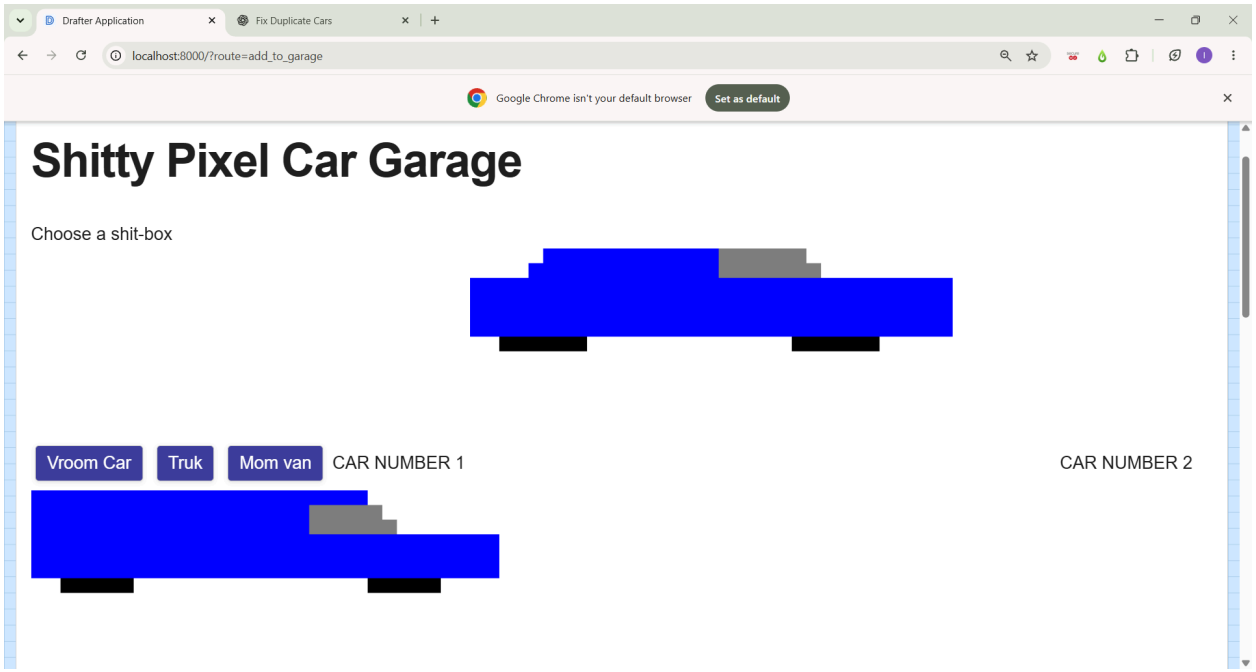
Your console shows two green trucks, even though your earlier example involved a blue sports car and a red truck.

I want to know whether you're accidentally clicking Save more than once, or whether the same car is being appended through another route.

One question: When you look at the website, does the text `CAR NUMBER 1` appear above or below the three buttons?

That answer will help us pinpoint the display issue without unnecessarily changing your working code.

Me:



Chat:

Okay, this screenshot finally reveals something important!

Your numbered labels are appearing in the correct order, but the pictures aren't following the same visual order.

I can see:

- `CAR NUMBER 1` appears immediately after the buttons.
- `CAR NUMBER 2` appears much farther to the right.
- The car pictures are being displayed in unexpected positions, including one above the buttons.

This suggests that the problem may be how Drafter lays out multiple `Picture` objects on a page, rather than how you're saving the cars.

Your `state.garage` loop is working: it's iterating through the saved cars and generating their pictures.

I checked the Drafter documentation, and there's a useful component we haven't tried yet: `Div`. It groups page content into a block-level container.

Drafter Documentation+1

I want to try grouping each car's label and picture together.

Change your loop to:

```
None
for i, car in enumerate(state.garage):
    content.append(
        Div(
            f"CAR NUMBER {i+1}",
            color_car(car, car.color)
        )
    )
```

This should make each car and its label a single block rather than separate page elements.

Why I'm suggesting this

In your screenshot, `CAR NUMBER 1` and `CAR NUMBER 2` are appearing on the same horizontal line. That suggests the individual elements aren't being grouped the way we want.

Using `Div` gives each saved car its own container.

Try this before changing anything else. I'd like to see whether the cars now appear consistently underneath the buttons.

Me: that worked. can you explain the idea behind the "f" in front of the string, so I can see the python logic behind it?

Chat:

The `f` in front of a string stands for **formatted string literal**, usually called an **f-string**.

It's a Python feature that lets you insert variable values directly into a string.

Let's break down your example:

```
None
f"CAR NUMBER {i+1}"
```

1. What does the `f` do?

Normally, if you write:

```
None
"CAR NUMBER i+1"
```

Python treats everything inside the quotation marks as literal text. It won't calculate anything.

But when you add `f`:

```
None
f"CAR NUMBER {i+1}"
```

Python recognizes the curly braces `{}` as an instruction to evaluate the expression inside them.

So if `i = 0`, Python evaluates:

```
None
i + 1
```

Which gives `1`.

The resulting string becomes:

```
None
CAR NUMBER 1
```

2. Why use `i+1` instead of just `i`?

Look at your loop:

```
None
for i, car in enumerate(state.garage):
```

Python's `enumerate()` gives you two things:

- `i`: the index (position) of the car.
- `car`: the actual Car object.

Python starts counting indexes at 0, not 1.

Iteration	i	i+1	Display
First car	0	1	CAR NUMBER 1
Second car	1	2	CAR NUMBER 2
Third car	2	3	CAR NUMBER 3

We add 1 because humans generally count from 1.

3. You can put other expressions inside f-strings

For example:

```
None
name = "Sports Car"
number = 3

print(f"My {name} is car number {number}")
```

Output:

```
None
My Sports Car is car number 3
```

You can even do calculations:

```
None
print(f"Next car number: {number + 1}")
```

Output:

```
None
Next car number: 4
```

4. The important concept

Think of an f-string as a regular string with little spaces where Python can insert calculated values.

- `f` tells Python to look for expressions.
- `{}` identifies the expression to evaluate.
- Everything outside `{}` stays ordinary text.

A little challenge: If your garage has 5 cars, what would this produce?

```
None
for i, car in enumerate(state.garage):
    print(f"Car {i * 2 + 1}")
```

Try tracing the values of `i` yourself. This is a great way to get comfortable with f-strings and loops!

Me:
it would print:
Car 1
Car 3

Chat:

Exactly right! You're calculating:

- First iteration: `0 * 2 + 1 = 1` → `Car 1`
- Second iteration: `1 * 2 + 1 = 3` → `Car 3`
- Third iteration: `2 * 2 + 1 = 5` → `Car 5`

One small clarification: if your garage has 5 cars, it would print:

```
None
Car 1
Car 3
Car 5
Car 7
Car 9
```

You've got the f-string concept! The key is remembering that Python evaluates whatever expression is inside the curly braces separately for each iteration. 😊

NEW CHAT:

Me:
im trying to make an "add spoiler" option for my car app but state.drawn_car isn't updating i dont think. the car looks the same after "add_spoiler" is called. here is my code:

```
from drafter import *
from drafter import Picture
from dataclasses import dataclass
set_website_style("none")
@dataclass
class State:
    garage: list[Car]
    latest_car: Car
```

```
    drawn_car: Picture
    car_color: str
@dataclass
class Car:
    body: list[str]
    color: str
    name: str
MATERIALS = {
    "B": "body",
    "T": "tire",
    "W": "window"
    }
COLORS = {
    "Blue": "b",
    "Red": "r",
    "Green": "g",
    "Orange": "o",
    "Pink": "p",
    "Black": "x"
    }

CARS = {
"sports": [
"        BBBBBBBBBBBBBBB          ",
"      BBBBBBBBBBWWWWWWWW         ",
"    BBBBBBBBBBBBBWWWWWWWWWW",
"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBHHHH",
"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
"BBBBBBTTTTTBBBBBBBBBBBBTTTTTBBBBBB",
"  BBBTTTTTTTBBBBBBBBBBTTTTTTTBBB",
"     TTTTTTT           TTTTTTT",
"      TTTTT             TTTTT"
    ],
"truck": [
"                    BBBBBBBBBBBBB",
"                   BBBBBBBBBWWWWWBW",
"                   BBBBBBBBBWWWWWWBW",
"                   BBBBBBBBBWWWWWWWBW",
"                   BBBBBBBBBWWWWWWWWBW",
"  BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
" BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBHHHHH",
"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBHHHHH",
"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
```

```
"BBBBBBBTTTTTTBBBBBBBBBBBBBBBBBBBBBBBBBTTTTTTBBBBBBB",
"BBBBBBTTTTTTTTBBBBBBBBBBBBBBBBBBBBBBBBTTTTTTTTBBBBBB",
"     TTTTTTTT                  TTTTTTTT",
"     TTTTTTTT                  TTTTTTTT",
"      TTTTTT                   TTTTTT"    ],
"van": [
"  BBBBBBBBBBBBBBBBBBBBBBBBBBB",
" BBBBBBBBBBBBBBBBBBBBBBBBBBBB",
"BBBBBBBBBBBBBBBBBBBBBBBBWWWWWWWWWBB",
"BBBBBBBBBBBBBBBBBBBBBBBBWWWWWWWWWWBB",
"BBBBBBBBBBBBBBBBBBBBBBBWWWWWWWWWWWBB",
"BBBBBBBBBBBBBBBBBBBBBBBWWWWWWWWWWWBBB",
"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBHHHHH",
"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBHHHH",
"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
"BBBBBTTTTTBBBBBBBBBBBBBBBBBBBBTTTTTBBBBBB",
"    TTTTTTT                TTTTTTT",
"    TTTTTTT                TTTTTTT",
"     TTTTT                 TTTTT"
   ]
}

pix_ht = 10
pix_wd = 10

@route
def index(state: State)->Page:
   content = [
      Header("Shitty Pixel Car Garage"),
      "Choose a shit-box\n",
      Button("Vroom Car", "make_sports"),
      Button("Truk","make_truck"),
      Button("Mom van","make_van")
      ]
   for i, car in enumerate(state.garage):
      content.append(Div(
         color_car(car, car.color)))
      content.append(car.name)
   return Page(
      state, content)
@route
def make_sports(state: State):
```

```
    state.latest_car = Car(CARS["sports"],"","nameless shit-box")
    return Page(
        state, [
        "Choose a body paint color",
        RadioButtonGroup("car_color", ["Green","Blue","Red","Pink","Black","Orange"]),
        Button("Apply","choose_color")
        ])
@route
def make_truck(state: State):
    state.latest_car = Car(CARS["truck"],"","nameless shit-box")
    return Page(
        state, [
        "Choose a body paint color",
        RadioButtonGroup("car_color", ["Green","Blue","Red","Pink","Black","Orange"]),
        Button("Apply","choose_color")
        ])
@route
def make_van(state: State):
    state.latest_car = Car(CARS["van"],"","nameless shit-box")
    return Page(
        state, [
        "Choose a body paint color",
        RadioButtonGroup("car_color", ["Green","Blue","Red","Pink","Black","Orange"]),
        Button("Apply","choose_color")
        ])
@route
def choose_color(state: State, car_color:str):
    state.car_color = car_color
    state.latest_car.color = car_color
    state.drawn_car = color_car(state.latest_car,car_color)
    if state.latest_car.body == CARS["sports"]:
        return Page(
            state, [
            "NeEEeEEoOWWWWW\n",
            state.drawn_car,
            Div(Button("Save to garage","add_to_garage")),
            Div("You have created: "+state.latest_car.name+"\n",
            "Give it a name",
            TextBox("car_name"),
            Button("Change name","name_car")),
            Div(Button("Add a spoiler","add_spoiler"))
            ])
    elif state.latest_car.body == CARS["truck"]:
        return Page(
```

```
            state, [
            "now that's a TRUK\n",
            state.drawn_car,
            Div(Button("Save to garage","add_to_garage")),
            Div("You have created: "+state.latest_car.name+"\n",
            "Give it a name",
            TextBox("car_name"),
            Button("Change name","name_car"))
            ])
    else:
        return Page(
            state, [
            "wow so hip\n",
            state.drawn_car,
            Div(Button("Save to garage","add_to_garage")),
            Div("You have created: "+state.latest_car.name+"\n",
            "Give it a name",
            TextBox("car_name"),
            Button("Change name","name_car"))
            ])
def color_car(car:Car,car_color:str):
    if car.body == CARS["sports"]:
        car_canvas = Picture.new(800,120)
        scale = 10
    elif car.body == CARS["truck"]:
        car_canvas = Picture.new(600,150)
        scale = 8
    elif car.body == CARS["van"]:
        car_canvas = Picture.new(800,200)
        scale = 10
    for y, row in enumerate(car.body):
        for x, pixel in enumerate(row):
          for dy in range(scale):
              for dx in range(scale):
                  if pixel == "B":
                      car_canvas.set_pixel(x*scale+dx,y*scale+dy,car_color)
                  elif pixel == "W":
                      car_canvas.set_pixel(x*scale+dx,y*scale+dy,"gray")
                  elif pixel == "T":
                      car_canvas.set_pixel(x*scale+dx,y*scale+dy,"black")
                  elif pixel == "H":
                      car_canvas.set_pixel(x*scale+dx,y*scale+dy,"goldenrod")
                  else:
                      pass
```

```
    return car_canvas
@route
def add_to_garage(state: State):
    state.garage.append(state.latest_car)
    return index(state)
@route
def name_car(state: State, car_name:str):
    state.latest_car.name = car_name
    return Page(
            state, [
            "Successfully renamed!\n",
            state.drawn_car,
            Button("Save to garage","add_to_garage"),
            Div("Current name: "+state.latest_car.name+"\n",
            TextBox("car_name",state.latest_car.name),
            Button("Change name","name_car"))
            ])
@route
def add_spoiler(state:State):
    if state.latest_car.body == CARS["sports"]:
        for y, row in enumerate(state.latest_car.body):
            row = "            "+row
            if y == 3:
                row = "BBBBBBBBBB  "+row
            if y == 4:
                row = "BBBBBBBBBBB "+row
            if y == 5:
                row = "        BBB "+row
            if y == 6:
                row = "         BB "+row
    state.drawn_car = color_car(state.latest_car,state.latest_car.color)
    return Page(
            state, [
            "NeEEeEEoOWWWWW\n",
            state.drawn_car,
            Div(Button("Save to garage","add_to_garage")),
            Div("You have created: "+state.latest_car.name+"\n",
            "Give it a name",
            TextBox("car_name"),
            Button("Change name","name_car"))
            ])

start_server(State([],None,None,""))
```

Chat:

I see the problem! There are actually **two separate issues** in your `add_spoiler` function. One is preventing the spoiler from being added to the car's design, and the other is related to how Python handles strings.

Let's walk through them without rewriting your whole function.

1. You're not actually changing `state.latest_car.body`

Look at this part:

```
None
for y, row in enumerate(state.latest_car.body):
    row = "             "+row
    if y == 3:
        row = "BBBBBBBBBBB  "+row
```

The important thing to understand is that `row` is a **temporary variable**.

When you write:

```
None
row = "             "+row
```

you're creating a new string and assigning it to the local variable `row`.

You're NOT changing the original string inside `state.latest_car.body`.

Think about it this way:

- `state.latest_car.body` is your original list of rows.
- `row` is just a variable holding a reference to one of those strings.
- Reassigning `row` doesn't replace the corresponding item in the original list.

Guiding question: What Python operation could you use to replace an existing item in a list at a particular index?

(Hint: Think about how you would change `my_list[3]`.)

2. Your spoiler is being added to the wrong place

Look at this:

```
None
if y == 3:
    row = "BBBBBBBBBB  "+row
if y == 4:
    row = "BBBBBBBBBBB "+row
if y == 5:
    row = "         BBB "+row
```

You're adding characters to the beginning of the rows, which is a reasonable approach for drawing a spoiler.

However, notice that you're also adding 13 spaces to every row before checking its index.

That means you're shifting the entire car to the right.

One thing to consider: **Do you actually need to add spaces to every row, or could you modify only the rows where the spoiler needs to appear?**

3. Your redraw logic is correct!

This part is good:

```
None
state.drawn_car =
color_car(state.latest_car,state.latest_car.color)
```

You're doing exactly what you should:

1. Modify the car's design.
2. Pass the modified car into `color_car`.
3. Store the newly generated Picture in `state.drawn_car`.
4. Display the updated picture.

The problem is that step 1 isn't actually modifying the original car yet.

One more thing to keep in mind

Your `Car` dataclass stores:

```
None
body: list[str]
```

Strings are immutable in Python, meaning you can't change individual characters inside an existing string.

But you CAN replace an entire string at a particular position in your list.

That's an important distinction for your pixel-art project!

I'd fix issue #1 first. Once your original `body` list is actually being modified, your existing redraw logic should work. Try that and see what happens before changing anything else.

Me:
does this modified code look right:

```
@route
def add_spoiler(state:State):
    if state.latest_car.body == CARS["sports"]:
        for y, row in enumerate(state.latest_car.body):
            spaces = "            "
            state.latest_car.body[y]=spaces+state.latest_car.body[y]
            if y == 3:
                state.latest_car.body[y]="BBBBBBBBBB  "+state.latest_car.body[y]
            if y == 4:
                state.latest_car.body[y] = "BBBBBBBBBBB "+state.latest_car.body[y]
            if y == 5:
                state.latest_car.body[y] = "         BBB "+state.latest_car.body[y]
            if y == 6:
                state.latest_car.body[y] = "          BB "+state.latest_car.body[y]
    state.drawn_car = color_car(state.latest_car,state.latest_car.color)
```

Chat:

Yes! This is a big improvement. You're now modifying the actual list stored in `state.latest_car.body`, rather than just changing a temporary variable. That's exactly the issue we wanted to fix.

There are just two things I'd watch out for:

1. You're adding spaces twice on the spoiler rows

For example:

```
None
spaces = "              "
state.latest_car.body[y] = spaces + state.latest_car.body[y]

if y == 3:
    state.latest_car.body[y] = "BBBBBBBBBBB  " +
state.latest_car.body[y]
```

On row 3, you're adding:

- 13 spaces.
- 11 B's and 2 spaces.
- The original row.

That's perfectly valid Python, but make sure that's the positioning you intend.

2. Watch out for repeated clicks!

Every time `add_spoiler` runs, it adds another 13 spaces to every row.

So if someone clicks the button twice, the car will shift even farther to the right.

You might eventually want to prevent someone from adding the same spoiler more than once.

My recommendation

Correct approach
Your use of:

```
None
state.latest_car.body[y] = ...
```

is correct.

And this line is also correct:

```
None
state.drawn_car =
color_car(state.latest_car,state.latest_car.color)
```

One question to test your understanding: If you click "Add a spoiler" twice, what do you predict will happen to the car's position and spoiler?

I'd test that before moving on!

Me:
@route
def add_spoiler(state:State):
    if state.latest_car.body == CARS["sports"]:
        for y, row in enumerate(state.latest_car.body):
            spaces = "   "
            if y == 1:
                state.latest_car.body[y]="BBBBBBBBBB "+state.latest_car.body[y]
            if y == 2:
                state.latest_car.body[y] = "BBBBBBBBBBB "+state.latest_car.body[y]
            if y == 3:
                state.latest_car.body[y] = "   BBB "+state.latest_car.body[y]
            if y == 4:
                state.latest_car.body[y] = "   BB "+state.latest_car.body[y]
            else:
                state.latest_car.body[y] = spaces+state.latest_car.body[y]
    state.drawn_car = color_car(state.latest_car,state.latest_car.color)

Chat:

You're very close! Your positioning logic is improved, but there's one subtle Python issue with your `if` / `else` statements.

The problem is the `else`

Look at this section:

```
None
if y == 1:
    state.latest_car.body[y] = "BBBBBBBBBB
"+state.latest_car.body[y]
if y == 2:
    state.latest_car.body[y] = "BBBBBBBBBBB
"+state.latest_car.body[y]
if y == 3:
    state.latest_car.body[y] = "         BBB
"+state.latest_car.body[y]
if y == 4:
```

```
    state.latest_car.body[y] = "          BB
"+state.latest_car.body[y]
else:
    state.latest_car.body[y] = spaces+state.latest_car.body[y]
```

Python associates the `else` with the **closest preceding** `if`, which is `if y == 4`.

That means:

- When `y == 1`, you add the spoiler, but then also add spaces.
- When `y == 2`, you add the spoiler, but then also add spaces.
- When `y == 3`, you add the spoiler, but then also add spaces.
- When `y == 4`, you add the spoiler without extra spaces.
- All other rows get spaces.

Guiding question

What Python structure would let you check several conditions while having just ONE `else` for all the remaining rows?

Hint: It starts with `if`, followed by `elif` for the additional conditions.

That would let you separate the four spoiler rows from the rows that simply need shifting.

One other thing: your `spaces` variable is fine, and you're no longer adding spaces to the spoiler rows unnecessarily. You're on the right track!

NEW CHAT:

Me: what can i do to make my car app more interesting

Chat:

I think your pixel-art car customization app has a lot of potential! Right now, the core idea is that users choose a car and customize its appearance. You could make it feel more like an actual **car customization game** rather than just a program that draws cars.

Here are some ideas, ranging from relatively easy to more challenging.

1. Visual customization features

PixelWheels Porsche (pixel art car) | GameDev Market

Custom paint jobs

Let users choose from more colors, including metallic, matte, and two-tone paint jobs.

Easy

Toyota Matsukiyo TOM'S SUPRA JGTC Pixel Car Racer by MclarenF1GTRSOK on DeviantArt

Spoilers and body kits

You're already working on spoilers! You could add different spoiler styles, front bumpers, and side skirts.

Easy–medium

Variedade de roda de calota de carro retrô, png | PNGWing

Different wheels

Add different wheel designs, such as standard, racing, and oversized wheels.

Medium

60x60 Pixel Art Red Car with Rainbow Underglow | AI Art Generator | Easy-Peasy.AI

Lighting

Allow users to change headlight colors, add neon underglow, or tint windows.

Easy–medium

2. Make it interactive

These features could make the app feel much more complete.

- **Randomize button:** Generate a random car with random colors and accessories.
- **Save to garage:** Save customized cars so users can revisit their creations.
- **Compare cars:** Display two customized cars next to each other.
- **Car names:** Let users name their creations.
- **Reset button:** Return the car to its original appearance.

3. Add a game element

Pixel Car Racer - Best Racing Game for Car Enthusiasts

Build a garage

Users can collect multiple cars, each with its own customizations.

Play Pixel Car Racer on PC - Games.lol

Car challenges

Give users objectives like building the fastest-looking sports car or creating a car using only certain colors.

JDM Pixel Street Car Racing Price & Trophy Guide

Showroom mode

Let users display their cars against different backgrounds, such as a city, racetrack, or garage.

4. What I would prioritize

I wouldn't try to implement everything at once. I'd build it in stages:

Priority	Feature	Why
1	More spoiler designs	Builds on what you've already started
2	Randomize button	Makes the app fun to experiment with
3	Save cars to garage	Gives users a reason to keep customizing
4	Different wheel designs	Adds more visual variety
5	Background selection	Makes the finished cars look better

One idea I particularly like: Give users a customization menu where they can select a car, change its paint color, add accessories, and then save it to their garage.

That would also give you opportunities to practice Python concepts you're already using:

- Dictionaries for customization options.
- Lists for storing multiple cars.
- Dataclasses for car information.
- Conditionals for applying accessories.
- Functions for drawing each customization.

I'd be happy to help you develop one of these features step by step while leaving the actual coding decisions to you.

Which direction sounds most appealing: making it a more detailed car designer, or turning it into a car-collecting game?

Me:

im trying to add bigger wheels to my trucks by appending strings to the car then rearranging them, but they are rearranging correctly. Here is my code:

@route

def get_big_wheels(state:State):

state.latest_car.body.append("BBBBBBTTTTTTTTBBBBBBBBBBBBBBBBBBBBBBBBTTTTTTTTTBBBBBBB")

state.latest_car.body.append("BBBBBBTTTTTTTTBBBBBBBBBBBBBBBBBBBBBBBBTTTTTTTTTBBBBBBB")

    for y, row in enumerate(state.latest_car.body):

        if y=8:

            state.latest_car.body[y] = "BBBBBBBTTTTTTBBBBBBBBBBBBBBBBBBBBBBBBBTTTTTTBBBBBBBB"

        elif y=9:

            state.latest_car.body[y] = "BBBBBBTTTTTTTTBBBBBBBBBBBBBBBBBBBBBBBBTTTTTTTTBBBBBBB"

state.latest_car.body[10],state.latest_car.body[-1]=state.latest_car.body[-1],state.latest_car.body[10]

state.latest_car.body[11],state.latest_car.body[-1]=state.latest_car.body[-1],state.latest_car.body[11]

```
    state.drawn_car = color_car(state.latest_car,state.latest_car.color)

    return Page(

        state, [

        "that's better",

        state.drawn_car,

        Div(Button("Save to garage","add_to_garage")),

        Div("You have created: "+state.latest_car.name+"\n",

        "Give it a name",

        TextBox("car_name"),

        Button("Change name","name_car"))

        ])
```

Chat:

I see a few things that could be causing trouble. Your general approach of appending rows and then rearranging them makes sense, but there are some indexing issues worth looking at.

1. Your `if` statement has a syntax error

You wrote:

```
None
if y=8:
```

Python uses:

- `=` to assign a value.

- `==` to compare two values.

So this needs to be a comparison.

2. You're modifying the list while iterating through it

You append two rows before your loop, which is fine. However, remember that `enumerate(state.latest_car.body)` now includes those newly appended rows.

This means your `y` values might not correspond to the rows you originally intended to modify.

Question: How many rows does your truck body have before you append the two new wheel rows?

3. Your swapping operations

Look at these two lines:

```
None
state.latest_car.body[10],state.latest_car.body[-1]=state.latest_car.body[-1],state.latest_car.body[10]



state.latest_car.body[11],state.latest_car.body[-1]=state.latest_car.body[-1],state.latest_car.body[11]
```

Remember that `-1` always refers to the last element in the list.

After the first swap, the last element has changed. Therefore, the second swap is not necessarily moving the row you originally intended.

This is probably an important part of your problem.

4. One other thing to check

You're using:

```
None
state.drawn_car = color_car(state.latest_car,state.latest_car.color)
```

Make sure your `color_car` function is expecting those two arguments. In your earlier version, you were passing a `Picture` as well.

Let's debug this without rewriting your function.

First, tell me: how many rows are in your truck's `body` list before you append the two new rows?

Then we can work out exactly where each row is going and fix the rearrangement.

Me: 14

Chat:

Perfect. That means we can trace your list indexes.

Your truck starts with **14 rows**, indexed `0–13`.

After appending two rows, you have **16 rows**, indexed `0–15`.

Here's what that means for your swaps:

Index	What it refers to
8	Original row 8
9	Original row 9
10	Original row 10
11	Original row 11
13	Original last row
14	First appended row
15	Second appended row (`-1`)

Here's the issue

Your first swap exchanges indexes 10 and 15.

Your second swap exchanges indexes 11 and 15.

But remember, after the first swap, index 15 contains the original row from index 10.

So your second swap is moving that same row again.

Let's think through the intended result

If your goal is to insert two bigger-wheel rows at indexes 10 and 11, you want to move the original rows at indexes 10 and 11 out of the way.

Guiding question: What would happen if you swapped index 11 with index 14 first, and then index 10 with index 15?

Think about how swapping from the bottom upward might help preserve the rows you haven't moved yet.

Me:

I ran this just to see what happened and for some reason the whole car moves to the right?

```
@route

def get_big_wheels(state:State):


#state.latest_car.body.append("BBBBBBTTTTTTTTBBBBBBBBBBBBBBBBBBBBBBBBTTTTTTTTTBBBBBBB")


#state.latest_car.body.append("BBBBBBTTTTTTTTBBBBBBBBBBBBBBBBBBBBBBBBTTTTTTTTTBBBBBBB")

    for y, row in enumerate(state.latest_car.body):

        if y==8:

            state.latest_car.body[y] = "BBBBBBBTTTTTTBBBBBBBBBBBBBBBBBBBBBBBBBTTTTTTBBBBBBBB"

        elif y==9:
```

```
        state.latest_car.body[y] =
"BBBBBBTTTTTTTTBBBBBBBBBBBBBBBBBBBBBBBBTTTTTTTTBBBBBB"
    state.drawn_car = color_car(state.latest_car,state.latest_car.color)
    return Page(
        state, [
        "that's better",
        state.drawn_car,
        Div(Button("Save to garage","add_to_garage")),
        Div("You have created: "+state.latest_car.name+"\n",
        "Give it a name",
        TextBox("car_name"),
        Button("Change name","name_car"))
        ])
```

Chat:

I suspect the issue is the **number of leading spaces in your strings**.

Remember how your pixel-art drawing function works:

```
None
for x, pixel in enumerate(row):
```

The position of each character determines its horizontal pixel coordinate.

For example:

```
None
"     BBBBB"
```

starts drawing the car at x-coordinate 5.

But:

```
None
"BBBBB"
```

starts drawing at x-coordinate 0.

Looking at your new rows

Your replacement strings start immediately with `"B"`:

```
None
"BBBBBBBTTTTTTBBBBBBBBBBBBBBBBBBBBBBBBBBBTTTTTTBBBBBBBB"
```

If your original truck rows started with spaces, you've effectively shifted the artwork to the left within those rows.

However, if the *whole car* appears to move right, there's another possibility: your replacement rows may be wider than the original rows, changing the overall canvas dimensions or alignment.

Let's check something

Can you show me:

1. The original truck's rows 8 and 9.
2. The lengths of those original rows compared with your replacement strings.

You can use Python's `len()` to check string lengths.

I suspect we'll find the alignment issue there.

Me: 53 and 54

Chat:

That helps! Your replacement rows are 53 and 54 characters long.

The important question is whether the original rows are also that long.

Remember, your drawing function uses each character's index as its x-coordinate. If the rows have different lengths or different numbers of leading spaces, the car can become misaligned.

Can you check the lengths of the original rows 8 and 9?

Also, do the original rows start with spaces, or do they start immediately with `"B"`?

I want to establish whether the problem is row width or horizontal positioning before changing your code.

Me:

i feel like this should work but it doesnt

@route

def get_big_wheels(state:State):

state.latest_car.body.append("BBBBBBTTTTTTTTBBBBBBBBBBBBBBBBBBBBBBBBTTTTTTTTTBBBBBBB")

state.latest_car.body.append("BBBBBBTTTTTTTTBBBBBBBBBBBBBBBBBBBBBBBBTTTTTTTTTBBBBBBB")

    for y, row in enumerate(state.latest_car.body):

        if y==8:

            state.latest_car.body[y] = "BBBBBBBTTTTTTBBBBBBBBBBBBBBBBBBBBBBBBBTTTTTTBBBBBBBB"

        elif y==9:

            state.latest_car.body[y] = "BBBBBBTTTTTTTTBBBBBBBBBBBBBBBBBBBBBBBBTTTTTTTTBBBBBBB"

    swap_ind = -2

    for y, row in enumerate(state.latest_car.body):

        if y>10 and y<13:

            state.latest_car.body[swap_ind],state.latest_car.body[y] = state.latest_car.body[y],state.latest_car.body[swap_ind]

            swap_ind = swap_ind-1

        elif y>=13 and y<15:

state.latest_car.body[y],state.latest_car.body[y+1]=state.latest_car.body[y+1],state.latest_car.body[y]

```
    state.drawn_car = color_car(state.latest_car,state.latest_car.color)
    return Page(
        state, [
        "that's better",
        state.drawn_car,
        Div(Button("Save to garage","add_to_garage")),
        Div("You have created: "+state.latest_car.name+"\n",
        "Give it a name",
        TextBox("car_name"),
        Button("Change name","name_car"))
        ])
```

Chat:

I see what you're trying to do! You're attempting to move the original bottom rows around so that the two newly appended wheel rows fit underneath the truck.

Your approach is reasonable, but there are two problems with the indexing logic.

1. Your `swap_ind` logic

You start with:

```
None
swap_ind = -2
```

Since your list has 16 elements, `-2` refers to index 14.

Then, each time your loop runs, you decrement `swap_ind`.

Let's trace it:

y	swap_index	What gets swapped
11	-2 (14)	Index 14 and 11
12	-3 (13)	Index 13 and 12

So far, this is doing what you intend, assuming those are the rows you want to exchange.

2. Your second condition

```
None
elif y>=13 and y<15:


state.latest_car.body[y],state.latest_car.body[y+1]=state.latest_car.body[y+1],state.latest_car.body[y]
```

This is where I think things go wrong.

You're swapping:

- Index 13 with 14.
- Index 14 with 15.

But remember, you've already swapped some of those rows in the previous loop iterations.

You're effectively rearranging rows that have already been moved.

3. One important Python concept

You're using `enumerate()` while modifying the same list you're iterating through.

That's allowed, but it can make tracing the order of operations difficult.

I'd recommend temporarily printing your entire list with its indexes after each swap:

```
None
for i, row in enumerate(state.latest_car.body):

    print(i, row)
```

Put this immediately after each swapping operation.

That will let you see exactly where your wheel rows end up.

One question: Are you trying to achieve this final arrangement?

- Rows 0–9: Original truck (with modified wheel rows at 8 and 9).
- Rows 10–11: The two newly appended rows.
- Rows 12–13: The original bottom two rows.

If that's your intended result, we can simplify the rearrangement considerably without changing the way you're thinking about the problem.

Me: how can i change my color_car function so the car is centered on the canvas?

Chat:

The easiest way is to calculate a **horizontal offset** before drawing the car.

Right now, your `color_car` function probably uses the character's `x` position directly:

```
None
car_canvas.set_pixel(x, y, color)
```

Instead, you want to add an offset to `x`.

1. Calculate the horizontal offset

You'll need three things:

- Canvas width.
- Car width (the length of the longest row).
- The amount of empty space on either side.

The centering formula is:

$$offset = \frac{canvaswidth - carwidth}{2}$$

Since pixel coordinates need to be integers, use integer division (`//`).

2. Apply the offset

For example, if your canvas is 100 pixels wide and your car is 54 pixels wide:

$$(100 - 54)//2 = 23$$

That means the car should start drawing at x-coordinate 23.

Instead of using `x`, you'd use `x + offset` in your `set_pixel()` call.

3. One thing to consider

Since your truck rows might have different lengths, I'd recommend using the **longest row** to determine the car width:

```
None
car_width = max(len(row) for row in car.body)
```

This way, even when you add spoilers or bigger wheels, the entire car stays centered.

One question: What is the width of your `Picture` canvas? That will help us work out exactly where to put the offset in your existing function.

Me:

from drafter import *

from drafter import Picture

from dataclasses import dataclass

#set_website_title("Corgi Cafe")

#set_site_information(

    #author="Your Name",

    #description="Order snacks for very good dogs.",

```
    #sources="Thanks to the Drafter docs and my study group.",
    #planning="plan.pdf",
    #links=["https://github.com/your-username/your-repository"]
#)
#hide_debug_information()
#set_website_framed(False)
set_website_style("retro")
@dataclass
class State:
    garage: list[Car]
    latest_car: Car
    drawn_car: Picture
    car_color: str
@dataclass
class Car:
    body: list[str]
    color: str
    name: str
MATERIALS = {
    "B": "body",
    "T": "tire",
    "W": "window"
    }
COLORS = {
```

```
    "Blue": "b",
    "Red": "r",
    "Green": "g",
    "Orange": "o",
    "Pink": "p",
    "Black": "x"
    }

CARS = {
"sports": [
"       BBBBBBBBBBBBBBBB          ",
"     BBBBBBBBBBWWWWWWWWW         ",
"    BBBBBBBBBBBBBWWWWWWWWWWW",
"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBHHHH",
"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
"BBBBBBTTTTTBBBBBBBBBBBBBTTTTTBBBBBB",
"  BBBTTTTTTTBBBBBBBBBBBTTTTTTTBBB",
"     TTTTTTT           TTTTTTT",
"      TTTTT             TTTTT"
    ],
"truck": [
"                    BBBBBBBBBBBBB",
"                   BBBBBBBBBWWWWWBW",
```

"   BBBBBBBBBWWWWWWBW",

"   BBBBBBBBBWWWWWWWBW",

"   BBBBBBBBBWWWWWWWWBW",

" BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",

" BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBHHHHH",

"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBHHHHH",

"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",

"BBBBBBBTTTTTTBBBBBBBBBBBBBBBBBBBBBBBBTTTTTTBBBBBBB",

"BBBBBBTTTTTTTTBBBBBBBBBBBBBBBBBBBBBBBTTTTTTTTBBBBBB",

"   TTTTTTTT   TTTTTTTT",

"   TTTTTTTT   TTTTTTTT",

"   TTTTTT   TTTTTT"   ],

"van": [

" BBBBBBBBBBBBBBBBBBBBBBBBBBB",

" BBBBBBBBBBBBBBBBBBBBBBBBBBBBB",

"BBBBBBBBBBBBBBBBBBBBBBBBBWWWWWWWWWBB",

"BBBBBBBBBBBBBBBBBBBBBBBBBWWWWWWWWWBB",

"BBBBBBBBBBBBBBBBBBBBBBBBBWWWWWWWWWBB",

"BBBBBBBBBBBBBBBBBBBBBBBBBWWWWWWWWWBBB",

"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",

"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",

"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBHHHHH",

"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBHHHH",

"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",

```
"BBBBBTTTTTBBBBBBBBBBBBBBBBBBBBBBTTTTTBBBBBBB",
"    TTTTTTT                 TTTTTTT",
"    TTTTTTT                 TTTTTTT",
"     TTTTT                  TTTTT"
   ]
}


pix_ht = 10
pix_wd = 10


@route
def index(state: State)->Page:
    content = [
        Header("Shitty Pixel Car Garage"),
        "Choose a shit-box\n",
        Button("Vroom Car", "make_sports"),
        Button("Truk","make_truck"),
        Button("Mom van","make_van")
        ]
    for i, car in enumerate(state.garage):
        content.append(Div(
            color_car(car, car.color)))
        content.append(car.name)
    return Page(
```

```
        state, content)
@route
def make_sports(state: State):
    state.latest_car = Car(CARS["sports"],"","nameless shit-box")
    return Page(
        state, [
        "Choose a body paint color",
        RadioButtonGroup("car_color", ["Green","Blue","Red","Pink","Black","Orange"]),
        Button("Apply","choose_color")
        ])
@route
def make_truck(state: State):
    state.latest_car = Car(CARS["truck"],"","nameless shit-box")
    return Page(
        state, [
        "Choose a body paint color",
        RadioButtonGroup("car_color", ["Green","Blue","Red","Pink","Black","Orange"]),
        Button("Apply","choose_color")
        ])
@route
def make_van(state: State):
    state.latest_car = Car(CARS["van"],"","nameless shit-box")
    return Page(
        state, [
```

```
        "Choose a body paint color",
        RadioButtonGroup("car_color", ["Green","Blue","Red","Pink","Black","Orange"]),
        Button("Apply","choose_color")
        ])
@route
def choose_color(state: State, car_color:str):
    state.car_color = car_color
    state.latest_car.color = car_color
    state.drawn_car = color_car(state.latest_car,car_color)
    if state.latest_car.body == CARS["sports"]:
        return Page(
            state, [
            "NeEEeEEoOWWWWW\n",
            state.drawn_car,
            Div(Button("Save to garage","add_to_garage")),
            Div("You have created: "+state.latest_car.name+"\n",
            "Give it a name",
            TextBox("car_name"),
            Button("Change name","name_car")),
            Div(Button("Add a spoiler","add_spoiler"))
            ])
    elif state.latest_car.body == CARS["truck"]:
        return Page(
            state, [
```

```
            "now that's a TRUK\n",
            state.drawn_car,
            Div(Button("Save to garage","add_to_garage")),
            Div("You have created: "+state.latest_car.name+"\n",
            "Give it a name",
            TextBox("car_name"),
            Button("Change name","name_car")),
            Button("IT NEEDS BIGGER WHEELS","get_big_wheels")
            ])
    else:
        return Page(
            state, [
            "wow so hip\n",
            state.drawn_car,
            Div(Button("Save to garage","add_to_garage")),
            Div("You have created: "+state.latest_car.name+"\n",
            "Give it a name",
            TextBox("car_name"),
            Button("Change name","name_car"))
            ])
def color_car(car:Car,car_color:str):
    if car.body == CARS["sports"]:
        car_canvas = Picture.new(800,120)
        scale = 10
```

```
    offset = (800 - len(car.body[9])) // 2
elif car.body == CARS["truck"]:
    car_canvas = Picture.new(480,150)
    scale = 8
    offset = (480 - len(car.body[7])) // 2
elif car.body == CARS["van"]:
    car_canvas = Picture.new(800,200)
    scale = 10
    offset = (800 - len(car.body[7])) // 2
for y, row in enumerate(car.body):
    for x, pixel in enumerate(row):
        for dy in range(scale):
            for dx in range(scale):
                if pixel == "B":
                    car_canvas.set_pixel((x+offset)*scale+dx,y*scale+dy,car_color)
                elif pixel == "W":
                    car_canvas.set_pixel((x+offset)*scale+dx,y*scale+dy,"gray")
                elif pixel == "T":
                    car_canvas.set_pixel((x+offset)*scale+dx,y*scale+dy,"black")
                elif pixel == "H":
                    car_canvas.set_pixel((x+offset)*scale+dx,y*scale+dy,"goldenrod")
                else:
                    pass
return car_canvas
```

```
@route
def add_to_garage(state: State):
    state.garage.append(state.latest_car)
    return index(state)
@route
def name_car(state: State, car_name:str):
    state.latest_car.name = car_name
    return Page(
            state, [
            "Successfully renamed!\n",
            state.drawn_car,
            Button("Save to garage","add_to_garage"),
            Div("Current name: "+state.latest_car.name+"\n",
            TextBox("car_name",state.latest_car.name),
            Button("Change name","name_car"))
            ])
@route
def add_spoiler(state:State):
    if state.latest_car.body == CARS["sports"]:
        for y, row in enumerate(state.latest_car.body):
            spaces = "              "
            if y == 1:
                state.latest_car.body[y]="      BBBBB  "+state.latest_car.body[y]
            elif y == 2:
```

```
            state.latest_car.body[y] = "         BBB "+state.latest_car.body[y]
        elif y == 3:
            state.latest_car.body[y] = "          BBB"+state.latest_car.body[y]
        else:
            state.latest_car.body[y] = spaces+state.latest_car.body[y]
    state.drawn_car = color_car(state.latest_car,state.latest_car.color)
    return Page(
        state, [
        "NeEEeEEoOWWWWW\n",
        state.drawn_car,
        Div(Button("Save to garage","add_to_garage")),
        Div("You have created: "+state.latest_car.name+"\n",
        "Give it a name",
        TextBox("car_name"),
        Button("Change name","name_car"))
        ])
@route
def get_big_wheels(state:State):

state.latest_car.body.append("BBBBBBTTTTTTTTBBBBBBBBBBBBBBBBBBBBBBBBTTTTTTTTTBBBBBBB")

state.latest_car.body.append("BBBBBBTTTTTTTTBBBBBBBBBBBBBBBBBBBBBBBBTTTTTTTTTBBBBBBB")
    for y, row in enumerate(state.latest_car.body):
        if y==8:
```

```
            state.latest_car.body[y] =
"BBBBBBBTTTTTTBBBBBBBBBBBBBBBBBBBBBBBBBTTTTTTBBBBBBB"

        elif y==9:

            state.latest_car.body[y] =
"BBBBBBTTTTTTTTBBBBBBBBBBBBBBBBBBBBBBBTTTTTTTTBBBBBB"

    swap_ind = -2

    for y, row in enumerate(state.latest_car.body):

        if y>10 and y<13:

            state.latest_car.body[swap_ind],state.latest_car.body[y] =
state.latest_car.body[y],state.latest_car.body[swap_ind]

            swap_ind = swap_ind+1

            print(y,row)

        elif y>=13 and y<15:


state.latest_car.body[y],state.latest_car.body[y+1]=state.latest_car.body[y+1],state.latest_car.bo
dy[y]

            print(y,row)

    state.drawn_car = color_car(state.latest_car,state.latest_car.color)

    return Page(

            state, [

            "yeeeAAAHHhhhHH\n",

            state.drawn_car,

            Div(Button("Save to garage","add_to_garage")),

            Div("You have created: "+state.latest_car.name+"\n",

            "Give it a name",

            TextBox("car_name"),

            Button("Change name","name_car"))
```

])

start_server(State([],None,None,""))

Chat:

Looking at your full code, I can see why your centering sometimes works and sometimes doesn't. You're actually very close!

The main issue is that you're calculating the offset using a specific row:

```
None
offset = (480 - len(car.body[7])) // 2
```

For the truck, row 7 happens to be 53 characters long, but that doesn't necessarily represent the full width of the truck. Some other rows may be longer.

1. Calculate the actual car width

You can use Python's `max()` function to find the longest row:

```
None
car_width = max(len(row) for row in car.body)
```

This expression:

- Goes through every row in `car.body`.
- Calculates the length of each row.
- Finds the largest length.

This is especially useful because you're dynamically adding rows when customizing your cars.

2. Calculate the offset

Your current formula is:

```
None
offset = (480 - len(car.body[7])) // 2
```

The general idea is correct, but you need to account for the scale factor.

Since your canvas uses 8 pixels per character for the truck, the actual calculation should be:

```
None
offset = (480 - car_width * scale) // 2
```

Important: This offset is now measured in actual canvas pixels, not character positions.

3. Adjust your drawing coordinates

Currently, you have:

```
None
(x+offset)*scale+dx
```

Because your new offset is already measured in canvas pixels, you should not multiply it by `scale` again.

Think about how you would separate the two calculations:

- `x * scale` determines the pixel position of the character.
- `offset` moves the entire car horizontally.
- `dx` determines which pixel within the scaled character you're drawing.

Your goal is to combine those three values.

4. One additional thing about your app

I noticed something important in your customization functions.

Every time you press **IT NEEDS BIGGER WHEELS**, you're appending two more rows:

```
None
state.latest_car.body.append(...)

state.latest_car.body.append(...)
```

That means clicking the button multiple times will keep adding wheels.

You might eventually want to prevent users from applying the same customization more than once.

My recommendation: Fix the centering first, then test all three vehicles:

- Sports car
- Truck with standard wheels
- Truck with bigger wheels
- Sports car with spoiler

That will help you determine whether your centering calculation works dynamically rather than just for one particular design.

You're developing a nice little graphics engine here, not just a car app!